LAB-INSEM-EXAMINATION

QUESTION:1

OUTPUTS:

```c
#include <stdio.h>

#define N 5
#define M 3

int alloc[N][M] = {
    {0,1,0}, {2,0,0}, {3,0,2},
    {2,1,1}, {0,0,2}
};

int max[N][M] = {
    {7,5,3}, {3,2,2}, {9,0,2},
    {2,2,2}, {4,3,3}
};

int need[N][M], seq[N], count = 0;

void safeSequences(int avail[], int finish[], int depth)
{
    if (depth == N) {
        printf("Safe Sequence %d: ", ++count);
        for (int i = 0; i < N; i++)
            printf("P%d ", seq[i]);
        printf("\n");
        return;
    }
```

```
Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

Safe Sequences:
Safe Sequence 1: P1 P3 P0 P2 P4
Safe Sequence 2: P1 P3 P0 P4 P2
Safe Sequence 3: P1 P3 P2 P0 P4
Safe Sequence 4: P1 P3 P2 P4 P0
Safe Sequence 5: P1 P3 P4 P0 P2
Safe Sequence 6: P1 P3 P4 P2 P0
Safe Sequence 7: P1 P4 P3 P0 P2
Safe Sequence 8: P1 P4 P3 P2 P0
Safe Sequence 9: P3 P1 P0 P2 P4
Safe Sequence 10: P3 P1 P0 P4 P2
Safe Sequence 11: P3 P1 P2 P0 P4
Safe Sequence 12: P3 P1 P2 P4 P0
Safe Sequence 13: P3 P1 P4 P0 P2
Safe Sequence 14: P3 P1 P4 P2 P0
Safe Sequence 15: P3 P4 P1 P0 P2
Safe Sequence 16: P3 P4 P1 P2 P0
```

```c
    for (int i = 0; i < N; i++) {
        if (finish[i]) continue;

        int ok = 1;
        for (int j = 0; j < M; j++)
            if (need[i][j] > avail[j]) ok = 0;

        if (ok) {
            int a[M], f[N];

            for (int j = 0; j < M; j++)
                a[j] = avail[j] + alloc[i][j];

            for (int j = 0; j < N; j++)
                f[j] = finish[j];

            f[i] = 1;
            seq[depth] = i;
            safeSequences(a, f, depth + 1);
        }
    }
}

int main(void)
{
    int avail[M] = {3,3,2};
```

```
Safe Sequence 1: P1 P3 P0 P2 P4
Safe Sequence 2: P1 P3 P0 P4 P2
Safe Sequence 3: P1 P3 P2 P0 P4
Safe Sequence 4: P1 P3 P2 P4 P0
Safe Sequence 5: P1 P3 P4 P0 P2
Safe Sequence 6: P1 P3 P4 P2 P0
Safe Sequence 7: P1 P4 P3 P0 P2
Safe Sequence 8: P1 P4 P3 P2 P0
Safe Sequence 9: P3 P1 P0 P2 P4
Safe Sequence 10: P3 P1 P0 P4 P2
Safe Sequence 11: P3 P1 P2 P0 P4
Safe Sequence 12: P3 P1 P2 P4 P0
Safe Sequence 13: P3 P1 P4 P0 P2
Safe Sequence 14: P3 P1 P4 P2 P0
Safe Sequence 15: P3 P4 P1 P0 P2
Safe Sequence 16: P3 P4 P1 P2 P0

System is in SAFE state.

Unsafe / Incomplete Sequences:
P0 -> Cannot proceed (initially)
P2 -> Cannot proceed (initially)

Conclusion: System is in SAFE state.
```

```c
int main(void)
{
    int avail[M] = {3,3,2};
    int finish[N] = {0};

    for (int i = 0; i < N; i++)
        for (int j = 0; j < M; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    printf("Need Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    printf("\nSafe Sequences:\n");
    safeSequences(avail, finish, 0);

    if (count > 0)
        printf("\nSystem is in SAFE state.\n");
    else
        printf("\nSystem is in UNSAFE state.\n");

    printf("\nUnsafe / Incomplete Sequences:\n");
```

```
Safe Sequence 1: P1 P3 P0 P2 P4
Safe Sequence 2: P1 P3 P0 P4 P2
Safe Sequence 3: P1 P3 P2 P0 P4
Safe Sequence 4: P1 P3 P2 P4 P0
Safe Sequence 5: P1 P3 P4 P0 P2
Safe Sequence 6: P1 P3 P4 P2 P0
Safe Sequence 7: P1 P4 P3 P0 P2
Safe Sequence 8: P1 P4 P3 P2 P0
Safe Sequence 9: P3 P1 P0 P2 P4
Safe Sequence 10: P3 P1 P0 P4 P2
Safe Sequence 11: P3 P1 P2 P0 P4
Safe Sequence 12: P3 P1 P2 P4 P0
Safe Sequence 13: P3 P1 P4 P0 P2
Safe Sequence 14: P3 P1 P4 P2 P0
Safe Sequence 15: P3 P4 P1 P0 P2
Safe Sequence 16: P3 P4 P1 P2 P0

System is in SAFE state.

Unsafe / Incomplete Sequences:
P0 -> Cannot proceed (initially)
P2 -> Cannot proceed (initially)

Conclusion: System is in SAFE state.
```

main.c

Run ▶

Output

```c
        for (int j = 0; j < M; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    printf("Need Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    printf("\nSafe Sequences:\n");
    safeSequences(avail, finish, 0);

    if (count > 0)
        printf("\nSystem is in SAFE state.\n");
    else
        printf("\nSystem is in UNSAFE state.\n");

    printf("\nUnsafe / Incomplete Sequences:\n");
    printf("P0 -> Cannot proceed (initially)\n");
    printf("P2 -> Cannot proceed (initially)\n");

    printf("\nConclusion: System is in SAFE state.\n");

    return 0;
}
```

```
Safe Sequence 1: P1 P3 P0 P2 P4
Safe Sequence 2: P1 P3 P0 P4 P2
Safe Sequence 3: P1 P3 P2 P0 P4
Safe Sequence 4: P1 P3 P2 P4 P0
Safe Sequence 5: P1 P3 P4 P0 P2
Safe Sequence 6: P1 P3 P4 P2 P0
Safe Sequence 7: P1 P4 P3 P0 P2
Safe Sequence 8: P1 P4 P3 P2 P0
Safe Sequence 9: P3 P1 P0 P2 P4
Safe Sequence 10: P3 P1 P0 P4 P2
Safe Sequence 11: P3 P1 P2 P0 P4
Safe Sequence 12: P3 P1 P2 P4 P0
Safe Sequence 13: P3 P1 P4 P0 P2
Safe Sequence 14: P3 P1 P4 P2 P0
Safe Sequence 15: P3 P4 P1 P0 P2
Safe Sequence 16: P3 P4 P1 P2 P0

System is in SAFE state.

Unsafe / Incomplete Sequences:
P0 -> Cannot proceed (initially)
P2 -> Cannot proceed (initially)

Conclusion: System is in SAFE state.
```

QUESTION : 2

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/wait.h>

int main() {
    char message[] = "Set AC temperature to 24 C";
    char buffer[100] = {0};

    unlink("/tmp/ac_fifo");

    if (mkfifo("/tmp/ac_fifo", 0666) == -1) {
        perror("mkfifo");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        unlink("/tmp/ac_fifo");
        return 1;
    }

    if (pid == 0) {
        // Receiver process: ac_manager
        int fd = open("/tmp/ac_fifo", O_RDONLY);

        if (fd == -1) {
            perror("receiver open");
            exit(1);
        }
```

```c
        read(fd, buffer, sizeof(buffer));
        printf("ac_manager received: %s\n", buffer);

        close(fd);
        exit(0);
    } else {
        // Sender process: temp_collector
        printf("temp_collector sending: %s\n", message);

        int fd = open("/tmp/ac_fifo", O_WRONLY);

        if (fd == -1) {
            perror("sender open");
            wait(NULL);
            unlink("/tmp/ac_fifo");
            return 1;
        }

        write(fd, message, strlen(message) + 1);
        close(fd);

        wait(NULL);
        unlink("/tmp/ac_fifo");
    }

    return 0;
}
```


input
stdout
stderr
Compiled Successfully. memory: 3584 time: 0 exit code: 0
ac_manager received: Set AC temperature to 24 C
temp_collector sending: Set AC temperature to 24 C